



PYTHON • DATA ANALYSIS

Знайомство з pandas

Бібліотека №1 для аналізу даних у Python

Лекція • 1.5 години

Що таке pandas?

Excel, але потужніший і автоматизований

pandas — це бібліотека Python для роботи з табличними даними. Якщо ви знаєте Excel чи Google Sheets — ви вже розумієте головну ідею.



Швидко

Обробляє мільйони рядків за секунди



Автоматизовано

Усе кодом, без ручних кліків мишкою



Універсально

CSV, Excel, бази даних, графіки, ML

Головний об'єкт pandas — DataFrame (таблиця з рядками і стовпцями)

План лекції

Від завантаження даних до групування



Завантаження даних

`read_csv`, перший погляд



Фільтрація

вибір рядків за умовами



Сортування

`sort_values`



Вибір стовпців

колонки і базові агрегати



Нові стовпці

обчислення на льоту



Групування

`groupby` — серце аналітики

Наші дані: мережа кав'ярень

360 замовлень за 6 місяців у 4 містах

order_id	city	product	revenue	manager
1255	Львів	Лате	140.0	Олена
1299	Харків	Чай	76.0	Богдан
1003	Харків	Маффін	60.0	Олена

Кожен рядок — одне замовлення. Столпці: місто, продукт, категорія, ціна, кількість, знижка, виручка, менеджер, дата.

Завантаження даних

read_csv та перший погляд

```
import pandas as pd

# Читаємо CSV-файл у DataFrame
df = pd.read_csv("coffee_sales.csv")

df.head()      # перші 5 рядків
df.shape       # (рядки, стовпці)
df.info()      # типи даних, пропуски
df.describe()  # статистика чисел
```

Перші кроки

.head()	перші рядки
.shape	розмір таблиці
.info()	огляд стовпців
.describe()	статистика

Вибір стовпців

Один стовпець або кілька

```
# Один стовпець
df["city"]

# Кілька стовпців (подвійні дужки!)
df[["product", "revenue"]]

# Агрегати по стовпцю
df["revenue"].sum()      # сума
df["revenue"].mean()     # середнє
df["revenue"].max()      # максимум

# Унікальні значення
df["city"].value_counts()
```



Запам'ятайте

Одинарні дужки `df["city"]` →
один стовпець (Series)

Подвійні дужки `df[[...]]` →
кілька стовпців (DataFrame)

Фільтрація рядків

Вибираємо тільки потрібні дані

```
# Тільки Київ
df[df["city"] == "Київ"]

# Виручка більше 200
df[df["revenue"] > 200]

# Дві умови: & (І), | (АБО)
df[(df["city"] == "Київ") &
    (df["revenue"] > 150)]
```



Часта помилка

Використовуйте **&** та **|**, НЕ **and/or**.

Кожну умову беріть у круглі дужки!

Принцип: усередині df[...] пишемо умову, яка дає True/False для кожного рядка

Створення нових стовпців

Обчислення на основі наявних даних

```
# Повна ціна = ціна × кількість  
df["full_price"] = df["unit_price"] * df["quantity"]  
  
# Скільки втрачено на знижці  
df["discount_amount"] = df["full_price"] - df["revenue"]
```



Векторизація — суперсила pandas

Операція застосовується одразу до ВСЬОГО стовпця — не треба писати цикл! pandas сам пройде по всіх рядках. Це і швидше, і коротше.

Сортування

sort_values впорядковує таблицю

```
# Топ-5 найбільших замовлень
df.sort_values("revenue",
               ascending=False).head() # або tail() для останніх 5-ти рядків

# Найдешевші замовлення
df.sort_values("revenue",
               ascending=True).head()
```

ascending=False

від найбільшого до найменшого

ascending=True

від найменшого до найбільшого



groupby()

Серце аналітики даних — те саме, що Pivot Table в Excel

Як працює groupby

Розбити → порахувати → об'єднати

```
df.groupby("city")["revenue"].sum()
```

1. За чим групуємо

"city"

2. Що рахуємо

"revenue"

3. Який агрегат

.sum()

Агрегати: .sum() .mean() .count() .max() .min() .agg([...])

Можна групувати і за кількома стовпцями: `df.groupby(["city", "category"])`

Приклади groupby

Відповідаємо на бізнес-питання

```
# Виручка по містах
df.groupby("city")["revenue"].sum()

# Топ продуктів за виручкою
df.groupby("product")["revenue"] \
    .sum().sort_values(ascending=False)

# Кілька агрегатів одразу
df.groupby("city")["revenue"] \
    .agg(["sum", "mean", "count"])
```

РЕЗУЛЬТАТ

city	
Київ	15465.0
Львів	14726.0
Одеса	12988.5
Харків	13588.5

Робота з датами

Виділяємо місяць, рік, день тижня

```
# Перетворюємо текст на справжню дату
df["date"] = pd.to_datetime(df["date"])

# Тепер виділяємо частини дати
df["month"] = df["date"].dt.month
df["day_name"] = df["date"].dt.day_name()

# Виручка по місяцях
df.groupby("month")["revenue"].sum()
```



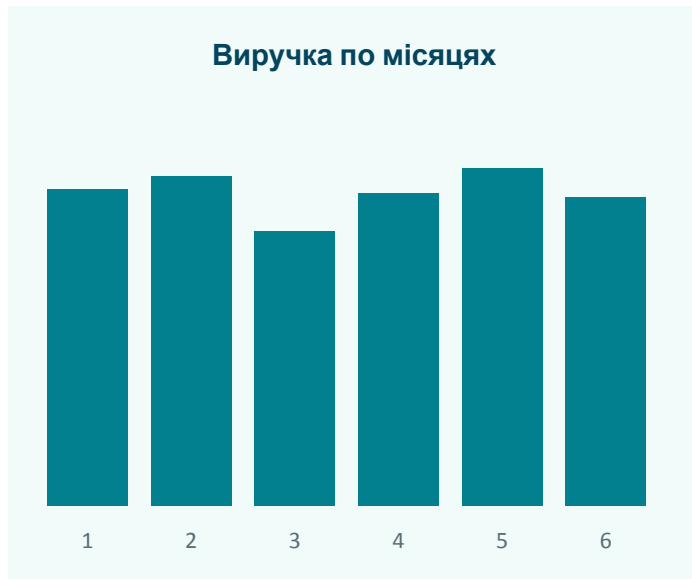
Після to_datetime доступні: .dt.year, .dt.month, .dt.day, .dt.day_name() та інші

Швидкі графіки

pandas вміє візуалізувати одразу

```
# Виручка по місяцях у вигляді графіка
monthly = df.groupby("month")["revenue"].sum()

monthly.plot(kind="bar",
              title="Виручка по місяцях")
```

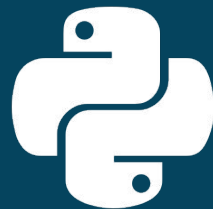


Для швидкого погляду — .plot() ідеальний. Для серйозної візуалізації — matplotlib та seaborn.

Шпаргалка

Усе найважливіше на одному слайді

<code>pd.read_csv(f)</code>	завантажити дані	<code>df["new"]=...</code>	новий стовпець
<code>df.head()/df.tail()</code>	перші/останні рядки	<code>df.sort_values()</code>	сортувати
<code>df.info()</code>	огляд таблиці	<code>df.groupby()</code>	групувати
<code>df["col"]</code>	вибрати стовпець	<code>.sum() .mean()</code>	агрегати
<code>df[df["x"]>5]</code>	фільтрувати	<code>.plot()</code>	графік



Час практики!

Відкрийте notebook і виконайте 10 завдань

Далі: merge (об'єднання таблиць), pivot_table, обробка пропусків, matplotlib

Дякую за увагу! 🐼